

COMP3000 Week 2

Little Languages + The Pipeline

COMP3000 Programming Languages



Last week in 60 seconds

- ▶ A **domain-specific language** (DSL) is a little language built for one job
- ▶ Examples: SQL, regex, LaTeX, Make, CSS, shell
- ▶ When you can, **reuse an existing one** instead of rolling your own
- ▶ The big idea: you will **build your own** little language for water flows
- ▶ The pipeline, teased: **Scanner** → **Parser** → **Interpreter**



From teaser to full picture

- ▶ Last week: just the names – Scanner, Parser, Interpreter
- ▶ The real pipeline has more stages than that
- ▶ Not every implementation walks the whole route
- ▶ Today: the **complete map**, then where our implementation stops



Little languages, again

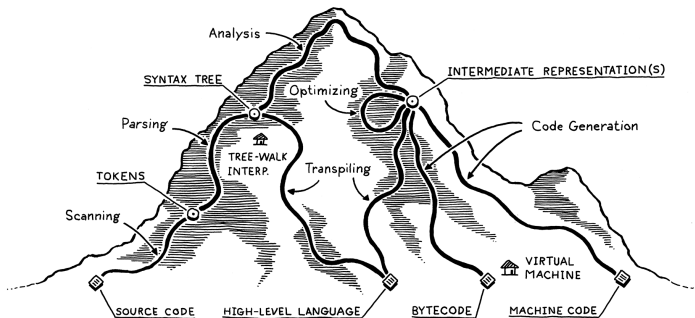
- ▶ Ch 1: “For every successful general-purpose language, there are a thousand successful niche ones”
- ▶ They are **pidgins tailor-built to a specific task**
- ▶ Small on purpose: restricted syntax, restricted features
- ▶ They lean on the machinery around them (their “host”) for the hard parts

Reuse... or roll your own?

- ▶ “When you can, good to reuse an existing one instead of rolling your own”
- ▶ Why: documentation, debuggers, editor support, syntax highlighting
- ▶ “Once you factor in all the other trappings, doing it yourself becomes a tall order”
- ▶ But when nothing fits your domain, building one is the right call – that’s this unit



The pipeline, in full



**Scan → Parse → Static Analysis → Optimise → Code Gen → VM
→ Runtime**

Front end, middle end, back end

- ▶ **Front end:** where source text becomes a structured representation
 - ▶ scanning and parsing
- ▶ **Middle end:** where we reason about the program
 - ▶ static analysis and optimisation
- ▶ **Back end:** where we produce something runnable
 - ▶ code generation, virtual machine, runtime

Static analysis: seeing what the code does

- ▶ The parser knows the *syntax*; static analysis works out the *meaning*
- ▶ **Binding**: connecting each identifier to the variable or function it refers to
- ▶ **Type checking**: rejecting programs that don't make sense (in typed languages)
- ▶ Ch 2: “get a high-level view of what the code is doing”
- ▶ In Lox: a **resolver** pass walks the tree before we evaluate it



Optimisation: doing less work

- ▶ If an expression always gives the same value, compute it once at build time
- ▶ **Constant folding:**

```
pennyArea = 3.14159 * (0.75 / 2) * (0.75 / 2);
```

becomes

```
pennyArea = 0.4417860938;
```

- ▶ “Optimization is a huge part of the programming language business”

Code generation: speaking the machine's language

- ▶ The back end converts our representation into runnable code
- ▶ **Native code:** instructions for a real chip (x86, ARM)
 - ▶ lightning fast, but tied to one architecture, and a lot of work
- ▶ **Bytecode:** instructions for a *virtual* machine
 - ▶ “a dense, binary encoding of the language’s low-level operations”
 - ▶ portable across architectures

Virtual machines and the runtime

- ▶ A **VM** emulates a hypothetical chip; it runs the bytecode at runtime
- ▶ Slower than native code, but simple and portable
 - ▶ “Implement your VM in C, and you can run your language on any platform with a C compiler”
- ▶ The **runtime** provides services while a program runs:
 - ▶ garbage collection, and tracking object types at runtime

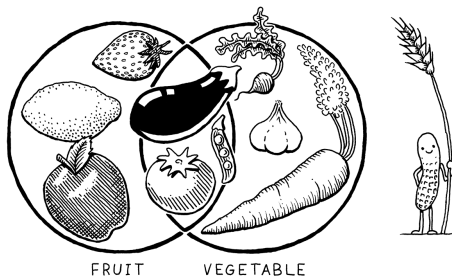


Where our journey stops

- ▶ Our first interpreter is a **tree-walk interpreter**: scan, parse, evaluate
 - ▶ no static analysis, no code generation, no VM – it walks the tree directly
- ▶ Later in the unit we add the static-analysis pass
- ▶ So we climb part of the mountain, and evaluate from the top

Compilers and interpreters

- ▶ “What’s the difference?” is like asking the difference between a fruit and a vegetable
- ▶ **Fruit** is a *botanical* term; **vegetable** is *culinary*
- ▶ Neither implies the negation of the other
- ▶ Tomatoes are both; apples are fruit but not vegetables; carrots are vegetables but not fruit

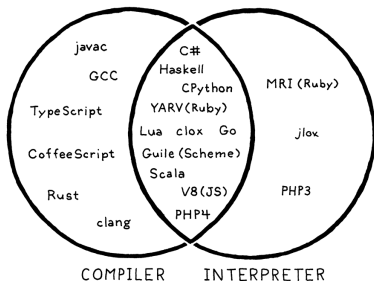


Compiling is an implementation technique

- ▶ **Compiling:** translating source to some other, usually lower-level, form
 - ▶ bytecode, machine code, or even another high-level language (transpiling)
- ▶ A **compiler** translates source but doesn't execute it – you run the output yourself
- ▶ An **interpreter** takes source and executes it directly, “from source”

It's not about the language

- ▶ **CPython**: runs your program from source (an interpreter), and it *has* a compiler – it parses and compiles to bytecode internally
- ▶ **Go**: go build compiles and stops; go run compiles *then* executes – it both *is* and *has* a compiler
- ▶ So “is Java compiled or interpreted?” is not meaningful
- ▶ We can only answer for **particular implementations**



Regex: a DSL for text

- ▶ **Regular expressions** are a little language for pattern matching in text
- ▶ Match a pattern against a string, e.g. `\d{4}-\d{2}-\d{2}` matches a date
- ▶ Look at its features and it is suspiciously like a general-purpose language:
 - ▶ **variables**: capture groups, reused later
 - ▶ **loops**: the `*` and `+` quantifiers
 - ▶ **conditions**: the `?` operator... sort of

- ▶ Head to **regexr.com** and try it yourself
- ▶ Try:
 - ▶ an exact string, e.g. “my presence”
 - ▶ one of two strings, e.g. “my presence” or “my friends”
 - ▶ a repeated letter, e.g. repeated p’s or g’s
 - ▶ capture the first word of each character’s dialogue

What makes a language complete?

- ▶ A language is **complete** if it can express all possible computations
- ▶ All complete languages have equivalent expressive power
- ▶ The **Turing view**: variables, conditions, and loops – like a Turing machine
- ▶ The **Church view**: variables and functions – like the lambda calculus
- ▶ “This dual nature of computing is one of the most fascinating aspects of programming language theory”

Wrap-up and Week 3 prep

- ▶ Today's reading: **Chapters 1–2** (re-read if needed)
- ▶ **Before next class:** read **Chapter 3** and complete the Week 3 iRAT
- ▶ Next week: **Lox** – the language we build, and its features
- ▶ Preview: a Lox program that drives turtle graphics